

Setup

```
import pandas as pd
import numpy as np
import plotly.express as px
import plotly.graph_objects as go

df = pd.read_csv('/content/global_superstore_cleaned.csv')
print(f"Loaded: {df.shape[0]:,} rows, {df.shape[1]} columns")
```

Loaded: 51,290 rows, 28 columns

Q1 — Which region–category combinations are quietly losing money despite high sales?

```
# Aggregate: profit margin (%) per Region x Category, plus total sales for context
agg = (df.groupby(['Region', 'Category'])
      .agg(Total_Sales=('Sales', 'sum'), Total_Profit=('Profit', 'sum'))
      .reset_index())
agg['Profit_Margin_%'] = (agg['Total_Profit'] / agg['Total_Sales'] * 100)

pivot = agg.pivot(index='Region', columns='Category', values='Profit_Margin_%')
pivot = pivot.loc[pivot.min(axis=1).sort_values().index]

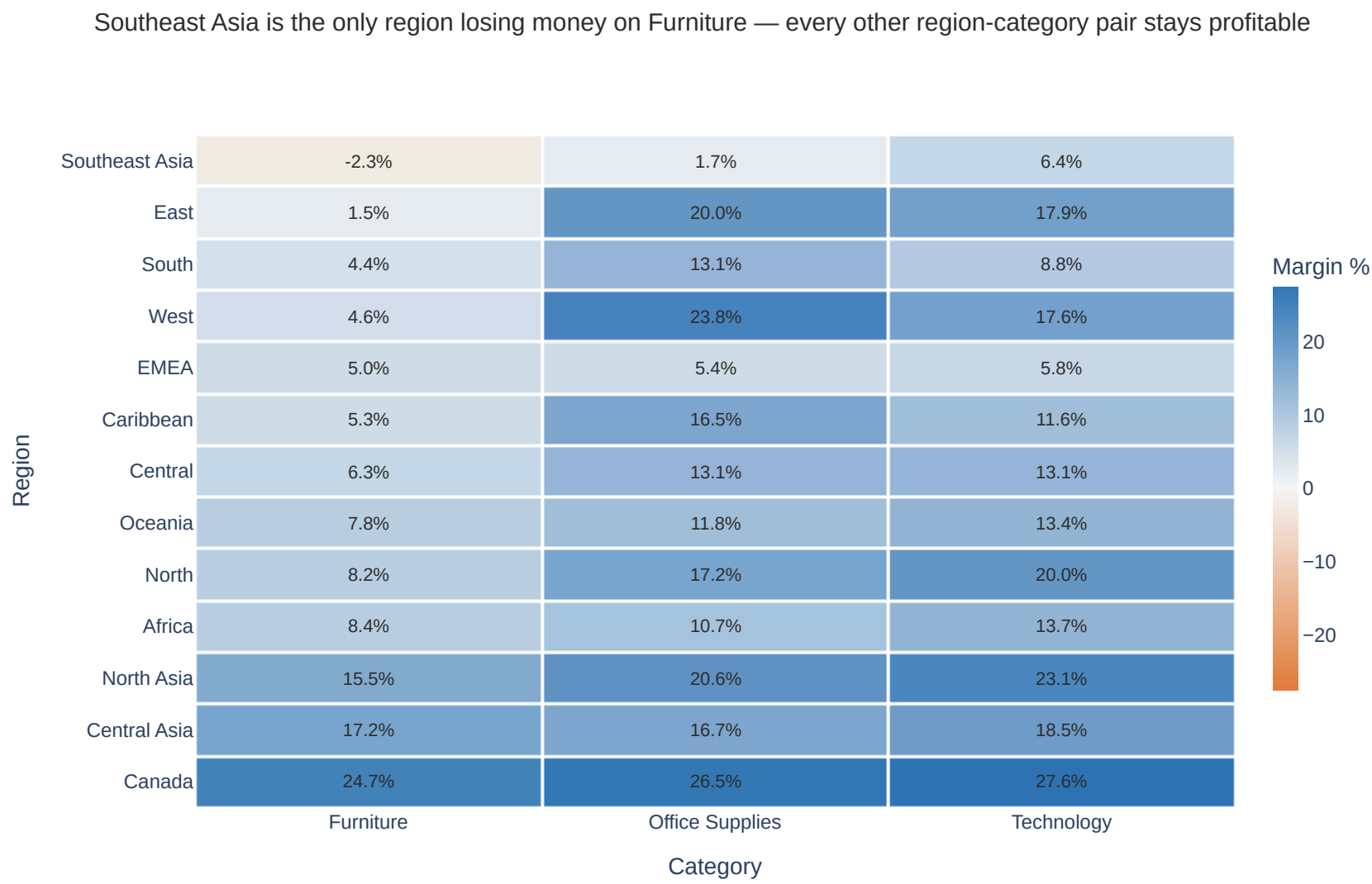
# CVD-safe diverging scale: orange (loss) -> white (breakeven) -> blue (profit)
# avoids red/green entirely, unlike default RdBu
cvd_diverging = [
    [0.0, '#E07B39'], # orange – loss
    [0.5, '#F5F5F5'], # near-white – breakeven
    [1.0, '#2E75B6']  # blue – profit
]

# — Step 1: Plotly Express base chart
fig = px.imshow(
    pivot,
    color_continuous_scale=cvd_diverging,
    color_continuous_midpoint=0,
    aspect='auto',
    text_auto='.1f',
    labels={'color': 'Profit Margin (%)'},
    height=550, width=850
)

# — Step 2: Customisation
fig.update_traces(
    texttemplate='%{z:.1f}%',
    textfont=dict(size=11, color='#2C2C2C'),
    hovertemplate='<b>{y}</b> – {x}<br>Profit Margin: %{z:.1f}%<extra></extra>',
    xgap=2, ygap=2 # thin gaps: Gestalt separation between cells
)

fig.update_layout(
    title=dict(
        text='Southeast Asia is the only region losing money on Furniture – every other region-category pair stays profitable',
        font=dict(family='Arial', size=15, color='#2C2C2C'),
        x=0.5, xanchor='center',
        y=0.97
    ),
    font=dict(family='Arial', size=12),
    plot_bgcolor='white', paper_bgcolor='white',
    coloraxis_colorbar=dict(title='Margin %', thickness=15, len=0.7),
    margin=dict(l=110, r=40, t=90, b=50),
    xaxis=dict(title='Category', side='bottom'),
    yaxis=dict(title='Region')
)

fig.show()
```



Q1 — Region × Category Profit Margin Heatmap

Question: Which region–category combinations are quietly losing money despite high sales?

Chart type & why A heatmap is used because we’re comparing a numeric measure (profit margin) across two categorical axes — Region and Category — simultaneously. A bar chart can’t show both dimensions at once without splitting into many small multiples.

Colour scale A custom **orange–white–blue diverging scale**, centred at 0% margin:

- Orange → loss

-  White → breakeven
-  Blue → profit

This preserves the diverging semantics while staying **CVD-safe** — no red/green relied on as the sole differentiator.

Sorting Regions are sorted by their *worst*-performing category margin, so the true problem region (Southeast Asia) surfaces at the top instead of being buried alphabetically.

Decluttering

- `text_auto` values shown in every cell — removes dependency on the colourbar for precision
- Thin `xgap`/`ygap` between cells for visual separation (Gestalt: proximity/enclosure)
- Clean white background, no gridlines

Insight Southeast Asia is the only region posting a net loss on Furniture (-2.3% margin) — every other region-category combination stays profitable, even if some margins (e.g. EMEA across the board) are thin.

Q2 — Does discount level affect profit margin across categories, and is there a tipping point where discounts destroy profit?

```
# Aggregate: average discount and profit margin per Sub-Category, sized by total sales
agg2 = (df.groupby(['Sub-Category', 'Category'])
        .agg(Avg_Discount=('Discount', 'mean'),
             Total_Sales=('Sales', 'sum'),
             Total_Profit=('Profit', 'sum'))
        .reset_index())
agg2['Profit_Margin_%'] = agg2['Total_Profit'] / agg2['Total_Sales'] * 100

# CVD-safe categorical palette for the 3 categories
color_map = {
    'Furniture': '#2E75B6',      # blue
    'Office Supplies': '#E07B39', # orange
    'Technology': '#4DAF4A'      # green (distinct enough from blue/orange, CVD-checked)
}

# — Step 1: Plotly Express base chart —————
fig = px.scatter(
    agg2,
    x='Avg_Discount', y='Profit_Margin_%',
    size='Total_Sales', color='Category',
    color_discrete_map=color_map,
    hover_name='Sub-Category',
    labels={'Avg_Discount': 'Average Discount', 'Profit_Margin_%': 'Profit Margin (%)'},
    size_max=55
)

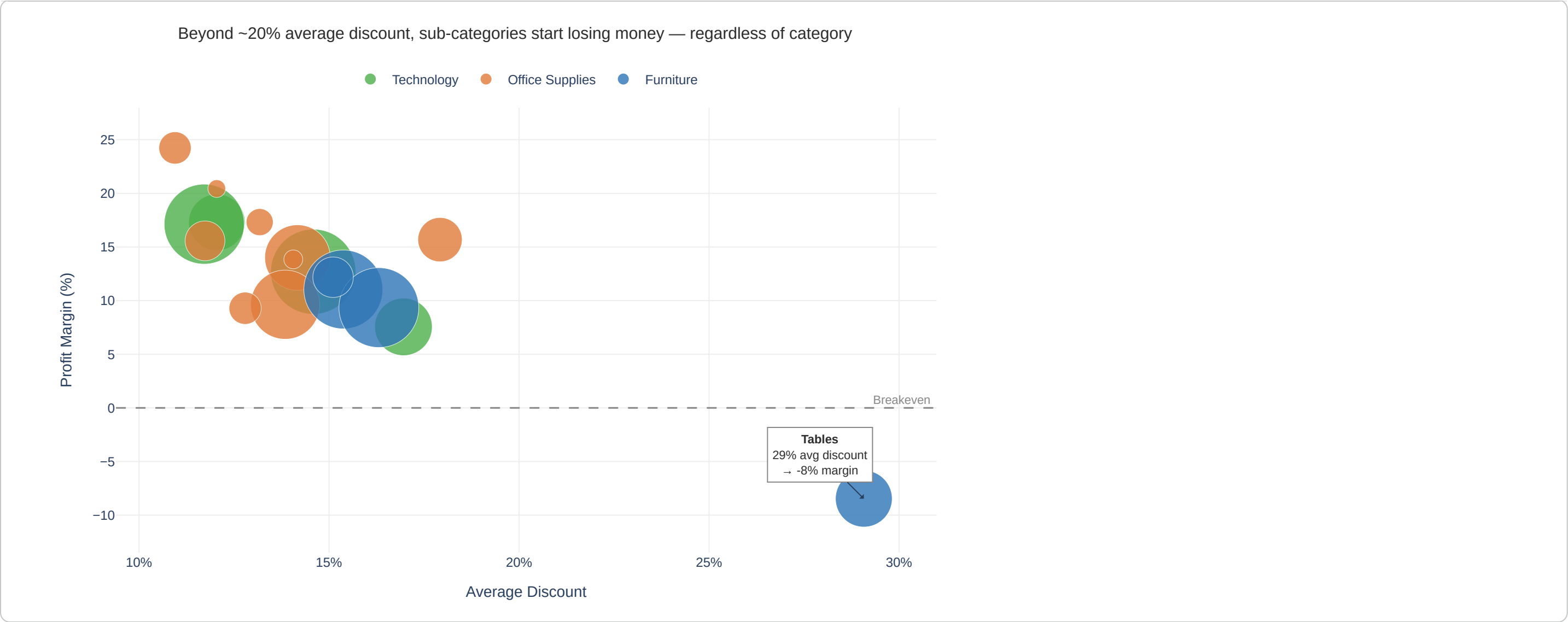
# — Step 2: Reference line – the tipping point where margin turns negative –
fig.add_hline(
    y=0, line_dash='dash', line_color='#888888', line_width=1.5,
    annotation=dict(text='Breakeven', font=dict(size=11, color='#888888'),
                    xanchor='left', yanchor='bottom', x=0.92)
)

# — Step 3: Annotate the clearest tipping-point example —————
worst = agg2.loc[agg2['Profit_Margin_%'].idxmin()]
fig.add_annotation(
    x=worst['Avg_Discount'], y=worst['Profit_Margin_%'],
    text=f"<b>{worst['Sub-Category']}</b><br>{worst['Avg_Discount']:.0%} avg discount<br>→ {worst['Profit_Margin_%']:.0f}% margin",
    showarrow=True, arrowhead=1, ax=-40, ay=-40,
    font=dict(size=11, family='Arial', color='#2C2C2C'),
    bgcolor='white', bordercolor='#888888', borderwidth=1, borderpad=4
)

# — Step 4: Customisation —————
fig.update_traces(marker=dict(opacity=0.8, line=dict(width=0.5, color='white'))

fig.update_layout(
    title=dict(
        text='Beyond ~20% average discount, sub-categories start losing money – regardless of category',
        font=dict(family='Arial', size=15, color='#2C2C2C'),
        x=0.5, xanchor='center', y=0.97
    ),
    font=dict(family='Arial', size=12),
    plot_bgcolor='white', paper_bgcolor='white',
    xaxis=dict(title='Average Discount', tickformat='.0%', gridcolor='#EEEEEE'),
    yaxis=dict(title='Profit Margin (%)', gridcolor='#EEEEEE', zeroline=False),
    legend=dict(title='', orientation='h', y=1.1, x=0.5, xanchor='center'),
    margin=dict(l=60, r=40, t=90, b=50),
    height=550, width=850
)

fig.show()
```



Q2 — Discount vs Profit Margin (Bubble Chart)

Question: Does discount level affect profit margin across product categories — is there a tipping point where discounts start destroying profit?

Chart type & why A bubble chart is used because we're relating three variables at once: average discount (x), profit margin (y), and total sales volume (bubble size) — while also distinguishing sub-categories by Category (colour). A simple bar or line couldn't carry this many dimensions cleanly.

Colour Categorical palette (blue / orange / green) encodes the 3 product Categories

Reference line A dashed **breakeven line at y=0%** gives immediate visual context — any bubble below the line is losing money outright, regardless of category.

Annotation Direct-labelled the clearest tipping-point example (**Tables**, 29% avg discount → -8% margin) rather than leaving the viewer to infer it from position alone.

Decluttering

- Light gridlines only, white background
- Legend moved above the plot, horizontal, out of the way of the data
- Bubble opacity + thin white border to handle overlap without adding visual noise

Insight Profit margin declines steadily as average discount increases, and beyond roughly 20% average discount, several sub-categories cross into negative margin — **Tables** being the clearest case, losing money outright at a 29% average discount.

Q3: How have sales and profit trended over time across customer segments — did any segment's growth stall or reverse?

```
# Aggregate: total sales per Segment per Year (using Year since Order Date is unusable)
trend = (df.groupby(['Year', 'Segment'])['Sales']
        .sum().reset_index())

# Identify which segment to highlight: compute growth rate 2011 -> 2014 per segment
growth = trend.pivot(index='Segment', columns='Year', values='Sales')
growth['pct_change'] = (growth[2014] - growth[2011]) / growth[2011] * 100
highlight_segment = growth['pct_change'].idxmax() # fastest growing segment
highlight_pct = growth.loc[highlight_segment, 'pct_change']

# Colour map: grey everything, highlight the fastest-growing segment
color_map = {seg: ('#2E75B6' if seg == highlight_segment else '#DDDDDD')
             for seg in trend['Segment'].unique()}

# — Step 1: Plotly Express base chart —————
fig = px.line(
    trend, x='Year', y='Sales', color='Segment',
    color_discrete_map=color_map,
    labels={'Sales': 'Total Sales ($)'},
    markers=True
)

# — Step 2: Line weight – highlighted segment thicker, others thin —————
fig.update_traces(line=dict(width=1.5), marker=dict(size=6), showlegend=False)
fig.update_traces(line=dict(width=3.5), marker=dict(size=8),
                  selector=dict(name=highlight_segment))

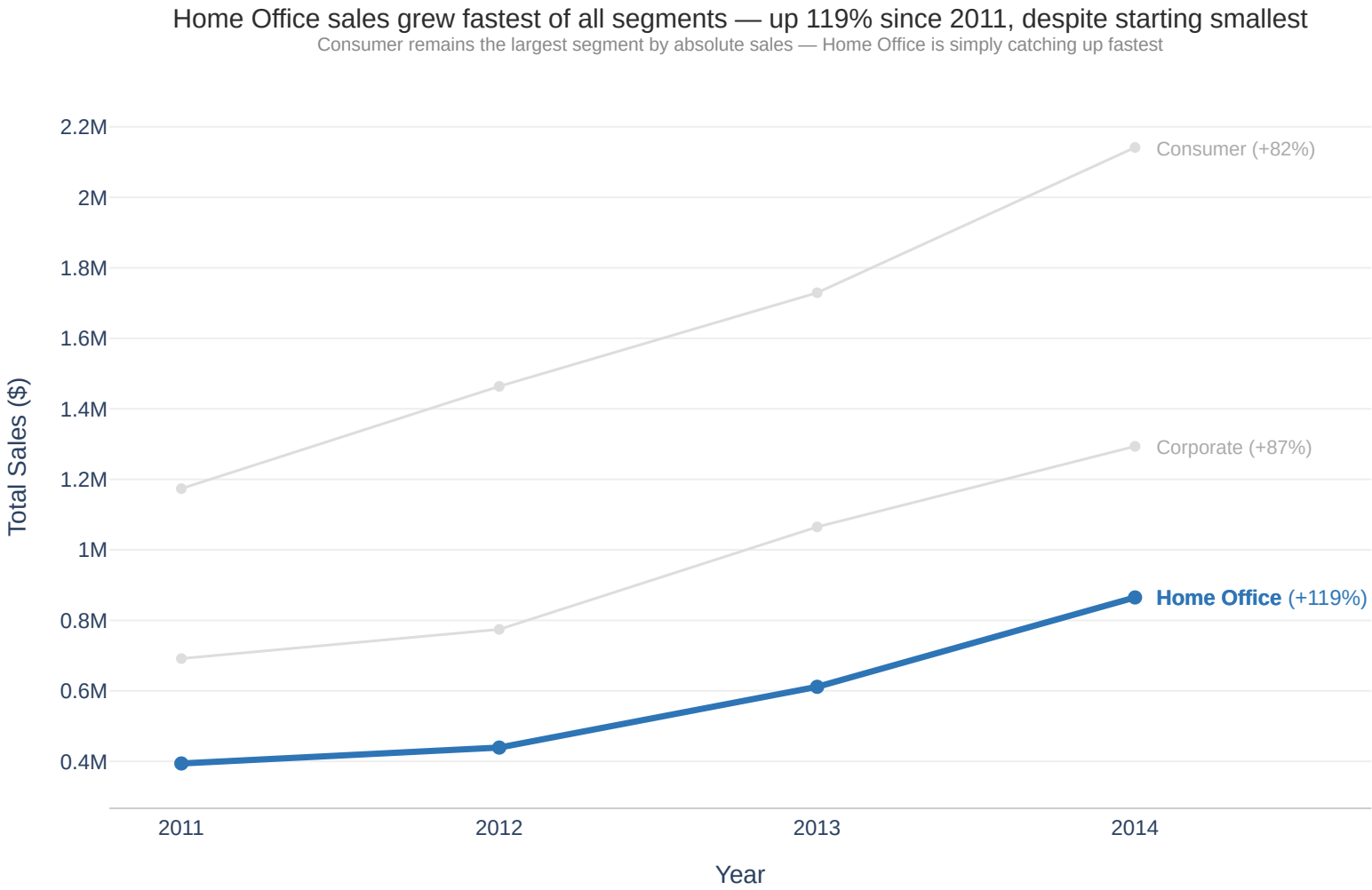
# — Step 3: Direct label at end of highlighted line —————
last_point = trend[(trend['Segment'] == highlight_segment) & (trend['Year'] == trend['Year'].max())]
fig.add_annotation(
    x=last_point['Year'].values[0], y=last_point['Sales'].values[0],
    text=f'<b>{highlight_segment}</b>', showarrow=False,
    xanchor='left', xshift=10,
    font=dict(color='#2E75B6', size=12, family='Arial')
)

# Also label the other (grey) segments directly at their endpoints, greyed
for seg in trend['Segment'].unique():
    pt = trend[(trend['Segment'] == seg) & (trend['Year'] == trend['Year'].max())]
    pct = growth.loc[seg, 'pct_change']
    is_highlight = seg == highlight_segment
    label = f'<b>{seg}</b> (+{pct:.0f}%)' if is_highlight else f'{seg} (+{pct:.0f}%)'
    fig.add_annotation(
        x=pt['Year'].values[0], y=pt['Sales'].values[0],
        text=label, showarrow=False, xanchor='left', xshift=10,
        font=dict(color='#2E75B6' if is_highlight else 'AAAAAA',
                  size=12 if is_highlight else 11, family='Arial')
    )
```



```
# — Step 4: Customisation —————
fig.update_layout(
    title=dict(
        text=(f'{highlight_segment} sales grew fastest of all segments — up {highlight_pct:.0f}% since 2011, despite starting smallest'
              f'<br><sup style="color:#888888">Consumer remains the largest segment by absolute sales — {highlight_segment} is simply catching up fastest</sup>'),
        font=dict(family='Arial', size=15, color='#2C2C2C'),
        x=0.5, xanchor='center', y=0.95
    ),
    font=dict(family='Arial', size=12),
    plot_bgcolor='white', paper_bgcolor='white',
    xaxis=dict(title='Year', showgrid=False, tickmode='array',
              tickvals=trend['Year'].unique(), showline=True, linecolor='#CCCCCC'),
    yaxis=dict(gridcolor='#EEEEEE', title='Total Sales ($)', zeroline=False),
    margin=dict(l=70, r=70, t=70, b=50), # slightly taller top margin for the subtitle line
    height=520, width=850
)

fig.show()
```



Q3 — Sales Trend by Customer Segment (Line Chart with Highlight)

Question: How have sales and profit trended over time across customer segments — did any segment's growth stall or reverse?

Chart type & why A line chart is used because Year is a genuine continuous trend, not a discrete category — the line correctly implies "this is a trajectory." With only 3 segments, a multi-series line avoids the spaghetti risk while still allowing comparison.

Highlight technique Grey-and-highlight: all three segments are plotted, but only the fastest-growing segment (by % change) is shown in bold blue with a thicker line. The other two recede into light grey.

Direct labelling Every segment is labelled directly at its line-end (bold for the highlight, grey for the rest) instead of a legend (Gestalt: proximity).

Title with context The main title states the headline finding (fastest % growth), but a grey subtitle line immediately qualifies it — Consumer is still the largest segment by absolute sales. This avoids a misleading takeaway: fast % growth off a small base isn't the same as market leadership, and the title says both without requiring the reader to dig into the data themselves.

Decluttering

- No vertical gridlines (years are self-explanatory)
- Light horizontal gridlines only, white background
- Explicit `tickvals` on x-axis to guarantee axis title renders correctly

Insight Home Office grew fastest in percentage terms (+119% since 2011) despite starting as the smallest segment — but Consumer remains the largest in absolute sales throughout, so the two findings coexist rather than contradict.

✓ Q4: Which sub-categories contribute disproportionately to profit vs. loss within each category?

```
# Aggregate total profit per Sub-Category, keep parent Category for grouping
sub_profit = (df.groupby(['Category', 'Sub-Category'])['Profit']
              .sum().reset_index()
              .sort_values('Profit'))

# Waterfall needs a running order + a final "Total" bar
labels = sub_profit['Sub-Category'].tolist() + ['Total']
values = sub_profit['Profit'].tolist() + [sub_profit['Profit'].sum()]
measures = ['relative'] * len(sub_profit) + ['total']

# — Step 1: Graph Objects – Waterfall isn't in Plotly Express —————
trace = go.Waterfall(
    x=labels,
    y=values,
    measure=measures,
    connector=dict(line=dict(color='#CCCCCC', width=1, dash='dot')),
    increasing=dict(marker_color='#2E75B6'), # blue – profitable sub-categories
    decreasing=dict(marker_color='#E07B39'), # orange – loss-making sub-categories (CVD-safe, not red)
    totals=dict(marker_color='#4D4D4D'),     # dark grey – net total
    texttemplate='%{y:$.0f}',
    textposition='outside',
    textfont=dict(size=10)
)

fig = go.Figure(data=[trace])

# — Step 2: Annotate the single worst-performing sub-category —————
worst_sub = sub_profit.iloc[0]
fig.add_annotation(
    x=worst_sub['Sub-Category'], y=worst_sub['Profit'],
    text=f"<b>{worst_sub['Sub-Category']}</b><br>${worst_sub['Profit']:,.0f} loss",
    showarrow=True, arrowhead=1, ax=0, ay=-50,
    font=dict(size=11, family='Arial', color='#E07B39'),
    bgcolor='white', bordercolor='#E07B39', borderwidth=1, borderpad=4
```



```
)

# — Step 3: Customisation —————
fig.update_layout(
    title=dict(
        text=f"{worst_sub['Sub-Category']} single-handedly drags down total profit – every other sub-category stays in the black",
        font=dict(family='Arial', size=15, color='#2C2C2C'),
        x=0.5, xanchor='center', y=0.96
    ),
    font=dict(family='Arial', size=12),
    plot_bgcolor='white', paper_bgcolor='white',
    xaxis=dict(title='Sub-Category', tickangle=-40, showgrid=False),
    yaxis=dict(title='Profit ($)', gridcolor='#EEEEEE', zeroline=False),
    showlegend=False,
    margin=dict(l=60, r=40, t=90, b=90),
    height=600, width=1000
)

fig.show()
```



Q4 — Sub-Category Profit Waterfall

Question: Which sub-categories contribute disproportionately to profit vs. loss within each category?

Chart type & why A waterfall chart shows how the total profit builds up sub-category by sub-category — unlike a bar chart, it visually accumulates each contribution into a running total, making the size of one bad performer relative to the whole immediately obvious.

Colour Blue = profitable sub-categories, orange = loss-making, dark grey = the final total bar.

Annotation Direct-labelled the single worst performer (Tables, -\$64,083) rather than leaving the viewer to spot the one orange bar unaided.

Decluttering

- Horizontal sub-category labels, light gridlines only, white background
- Dotted connector lines between bars keep the "running total" logic visible without adding heavy visual weight

Insight Tables is the only sub-category losing money (-\$64,083) — every other sub-category, including the rest of Furniture, stays profitable. This isolates the loss to one specific product line rather than an entire category.

✓ Q5: How does customer segment x region combination differ in average order value and profit margin?

```
seg_region = (df.groupby(['Segment', 'Region'])
               .agg(Avg_Order_Value=('Sales', 'mean'),
                   Total_Sales=('Sales', 'sum'),
                   Total_Profit=('Profit', 'sum'))
               .reset_index())
seg_region['Profit_Margin_%'] = seg_region['Total_Profit'] / seg_region['Total_Sales'] * 100

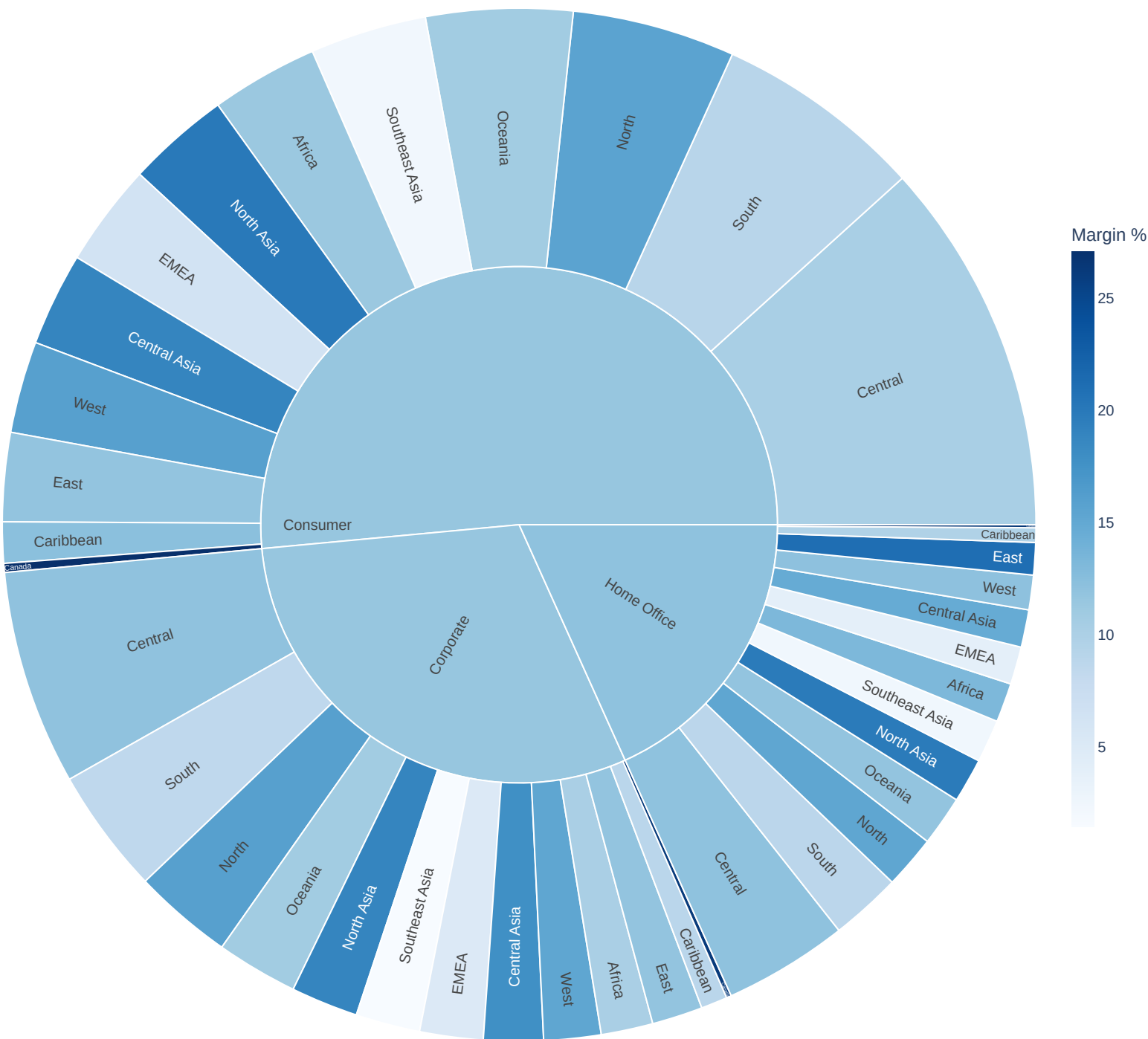
# — Step 1: Plotly Express base chart —————
fig = px.sunburst(
    seg_region,
    path=['Segment', 'Region'],
    values='Total_Sales',
    color='Profit_Margin_%',
    color_continuous_scale='Blues',          # sequential: all margins here are positive
    labels={'Profit_Margin_%': 'Profit Margin (%)'},
    height=800, width=800                    # bigger canvas – less label crowding
)

# — Step 2: Customisation —————
fig.update_traces(
    textinfo='label',                        # dropped 'percent parent' – was cluttering outer ring
    hovertemplate='<b>%(label)</b><br>Sales: $%(value:,.0f)<br>Margin: %(color:.1f)%<extra></extra>',
    insidetextorientation='radial',
    marker=dict(line=dict(color='white', width=1)) # thin white borders separate wedges (Gestalt)
)

fig.update_layout(
    title=dict(
        text='Consumer-Central is the biggest revenue pocket – but North Asia and Central Asia post the highest margins',
        font=dict(family='Arial', size=14, color='#2C2C2C'),
        x=0.5, xanchor='center', y=0.97
    ),
    font=dict(family='Arial', size=10),
    coloraxis_colorbar=dict(title='Margin %', thickness=15, len=0.6),
    margin=dict(l=10, r=10, t=80, b=10),
    paper_bgcolor='white'
```

```
)  
  
fig.show()
```

Consumer-Central is the biggest revenue pocket — but North Asia and Central Asia post the highest margins



Q5 — Segment × Region Spend & Margin (Sunburst)

Question: How does the customer segment x region combination differ in average order value and profit margin?

Chart type & why A sunburst is used because the data has genuine two-level hierarchy — Segment nests Region — and both the size (spend share) and a second metric (margin) need to be shown at once. A bar chart could show one level, not the nested relationship.

Colour Sequential Blues scale, chosen because every segment-region margin in this dataset is positive (1.5%–27%) — a diverging scale would be misleading here since there's no true "loss" side to diverge from.

Decluttering

- Dropped `percent parent` from labels — with 39 wedges, showing both name and percentage cluttered the outer ring; label-only is more readable
- Thin white borders between wedges (Gestalt: enclosure) to separate segments without needing a legend
- Enlarged canvas (800×800) to reduce text crowding on the outer ring

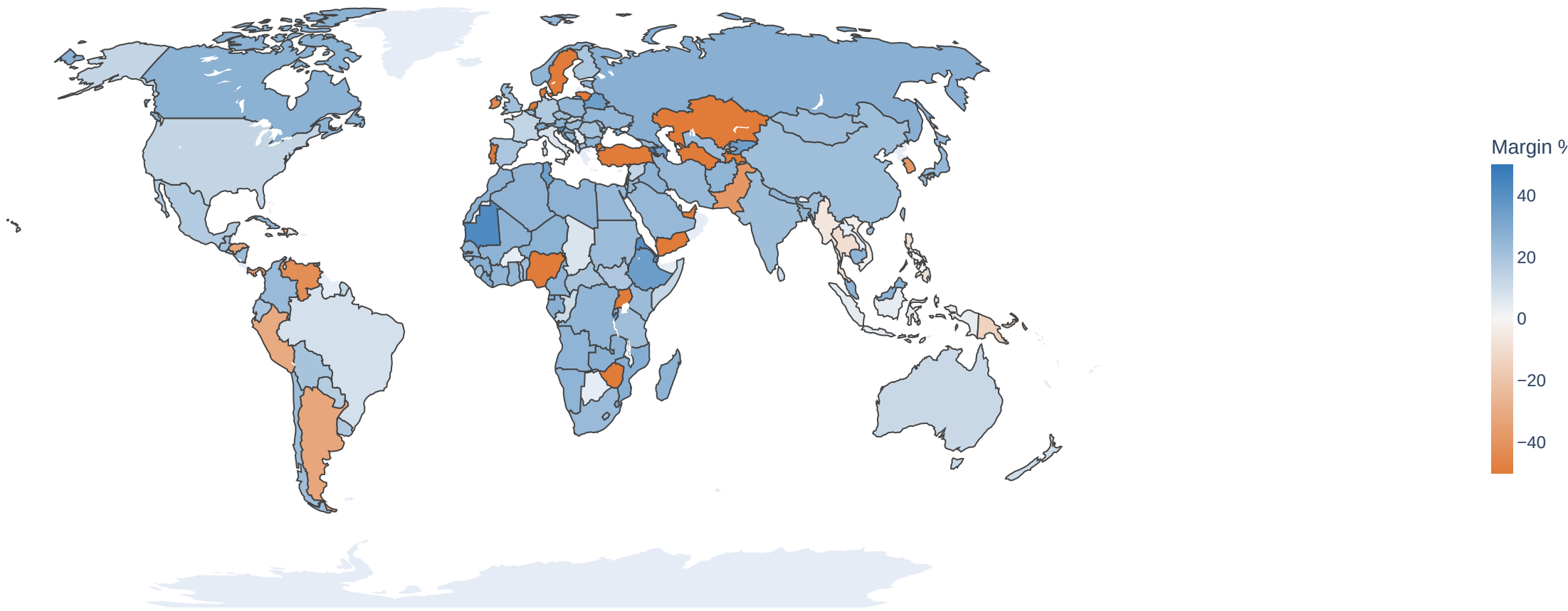
Insight Consumer-Central is the single biggest revenue pocket by far, but it's not the most profitable — North Asia and Central Asia post the highest margins across segments, showing that big spend and high margin don't necessarily overlap.

✓ Q6: Geographically, which countries drive the most profit — and which drive the most loss despite operating at real sales volume?

```
country_agg = (df.groupby('Country')  
                .agg(Total_Sales=('Sales', 'sum'), Total_Profit=('Profit', 'sum'))  
                .reset_index())  
country_agg['Profit_Margin_%'] = country_agg['Total_Profit'] / country_agg['Total_Sales'] * 100  
  
# A handful of tiny-volume countries produce extreme -100%+ margins (few orders,  
# one bad discount) – cap the colour range so they don't wash out the real signal  
color_range = [-50, 50]  
  
# — Step 1: Plotly Express base chart —————  
fig = px.choropleth(  
    country_agg,  
    locations='Country',  
    locationmode='country names',  
    color='Profit_Margin_%',  
    color_continuous_scale=[[0, '#E07B39'], [0.5, '#F5F5F5'], [1, '#2E75B6']], # CVD-safe diverging  
    color_continuous_midpoint=0,  
    range_color=color_range,  
    hover_name='Country',  
    hover_data={'Total_Sales': ':.0f', 'Profit_Margin_%': ':.1f', 'Country': False},  
    labels={'Profit_Margin_%': 'Profit Margin (%)'},  
    projection='natural earth'  
)  
  
# — Step 2: Customisation —————  
fig.update_layout(  
    title=dict(  
        text='Nigeria loses money despite $54K in sales – while major markets like China and India post the strongest margins (21%+)',  
        font=dict(family='Arial', size=14, color='#2C2C2C'),  
        x=0.5, xanchor='center', y=0.97  
    ),  
    font=dict(family='Arial', size=12),  
    geo=dict(showframe=False, showcoastlines=False, bgcolor='white'),  
    coloraxis_colorbar=dict(title='Margin %', thickness=15, len=0.6),  
    margin=dict(l=0, r=0, t=70, b=0),  
    paper_bgcolor='white',  
    height=500  
)
```

fig.show()

Nigeria loses money despite \$54K in sales — while major markets like China and India post the strongest margins (21%+)



Q6 — Country Profit Margin Choropleth

Question: Geographically, which countries drive the most profit — and which drive the most loss despite operating at real scale?

Chart type & why A choropleth is used because the geographic pattern itself is the finding — where profitability concentrates (or collapses) is more informative here than a ranked list, and the map gives immediate spatial context across 147 countries.

Colour CVD-safe diverging scale (orange = loss, blue = profit), centred at 0% margin — appropriate since this is genuinely an above/below-breakeven comparison.

Handling outliers A handful of very low-volume countries show extreme margins (e.g. -160%) driven by just 1–2 orders. The colour range was capped at ±50% so these statistical noise cases don't wash out the real, high-volume story.

Insight Nigeria loses money despite real sales volume (54K, −80K profit) — while major markets like China and India, both top-8 by sales, post the strongest margins (21%+). Scale and profitability aren't correlated here.

Q7: How does order quantity interact with discount to affect profitability — are big discounted bulk orders actually worth it?

```
df['Qty_Bin'] = pd.cut(df['Quantity'], bins=[0, 3, 6, 9, 14],
                        labels=['1-3', '4-6', '7-9', '10-14'])
df['Disc_Bin'] = pd.cut(df['Discount'], bins=[-0.01, 0.1, 0.2, 0.3, 0.5, 1.0],
                        labels=['0-10%', '10-20%', '20-30%', '30-50%', '50%+'])

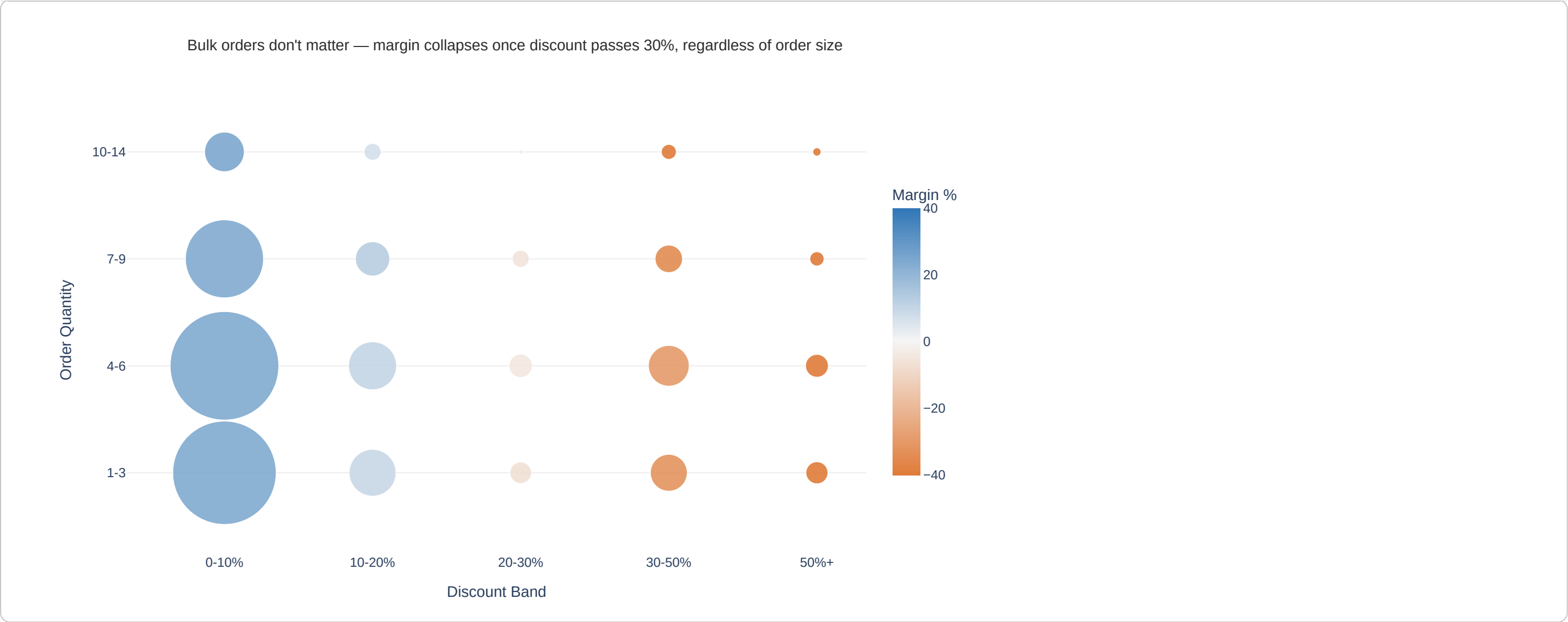
agg7 = (df.groupby(['Qty_Bin', 'Disc_Bin'], observed=True)
        .agg(Total_Sales=('Sales', 'sum'), Total_Profit=('Profit', 'sum'))
        .reset_index())
agg7['Margin'] = agg7['Total_Profit'] / agg7['Total_Sales'] * 100

# — Step 1: Plotly Express base chart —————
fig = px.scatter(
    agg7,
    x='Disc_Bin', y='Qty_Bin',
    size='Total_Sales', color='Margin',
    color_continuous_scale=[[0, '#E07B39'], [0.5, '#F5F5F5'], [1, '#2E75B6']], # CVD-safe diverging
    color_continuous_midpoint=0,
    range_color=[-40, 40],           # added – caps the scale so the 50%+ discount extreme
                                    # doesn't wash out the genuinely strong 0-10% margins
    hover_data={'Margin': ':.2f'},
    labels={'Disc_Bin': 'Discount Band', 'Qty_Bin': 'Order Quantity',
            'Margin': 'Profit Margin (%)'},
    size_max=70
)

# — Step 2: Customisation —————
fig.update_traces(marker=dict(line=dict(width=0.5, color='white'), opacity=0.9))

fig.update_layout(
    title=dict(
        text="Bulk orders don't matter – margin collapses once discount passes 30%, regardless of order size",
        font=dict(family='Arial', size=14, color='#2C2C2C'),
        x=0.5, xanchor='center', y=0.95
    ),
    font=dict(family='Arial', size=12),
    plot_bgcolor='white', paper_bgcolor='white',
    xaxis=dict(title='Discount Band', showgrid=False),
    yaxis=dict(title='Order Quantity', gridcolor='#EEEEEE'),
    coloraxis_colorbar=dict(title='Margin %', thickness=25, len=0.7),
    margin=dict(l=70, r=40, t=90, b=50),
    height=550, width=850
)

fig.show()
```

Q7 — Quantity × Discount Profitability Matrix (Bubble Chart)

Question: How does order quantity interact with discount to affect profitability — are big discounted bulk orders actually worth it?

Chart type & why A bubble chart on a Discount-Band × Quantity-Band grid is used to test two variables jointly against margin — bar or line charts can't show three dimensions (two categorical axes + a continuous colour + volume as size) at once.

Colour Diverging scale (orange = loss, blue = profit), centred at 0%. The colour range was explicitly capped at ±40% — without this, one extreme outlier bin (-112% margin at 50%+ discount) stretched the scale so far that genuinely healthy +23% margins appeared pale instead of solidly blue.

Bubble size Encodes total sales volume per bin — lets the viewer see which combinations are common vs rare, alongside the margin story.

Decluttering White background, no gridlines on the discount axis (categories are self-explanatory), light gridlines on quantity axis only.

Insight Order quantity barely affects margin — bubbles are roughly the same colour across each row. What actually destroys profit is discount: margin holds around +23% up to 20% discount, then collapses hard past 30%, regardless of whether it's a single unit or a bulk order. Bulk size doesn't protect margin.

Q8: Which product sub-categories show the most volatile sales patterns across years, and which are the most stable?

```
sub_year = df.groupby(['Sub-Category', 'Year'])['Sales'].sum().reset_index()

# Volatility = coefficient of variation (std / mean) across the 4 years per sub-category
pivot = sub_year.pivot(index='Sub-Category', columns='Year', values='Sales')
cv = (pivot.std(axis=1) / pivot.mean(axis=1)).sort_values(ascending=False)

most_volatile = cv.index[0]
most_stable = cv.index[-1]

# Colour map: grey everything, highlight the most volatile (orange) and most stable (blue)
color_map = {sc: '#DDDDDD' for sc in sub_year['Sub-Category'].unique()}
color_map[most_volatile] = '#E07B39'
color_map[most_stable] = '#2E75B6'

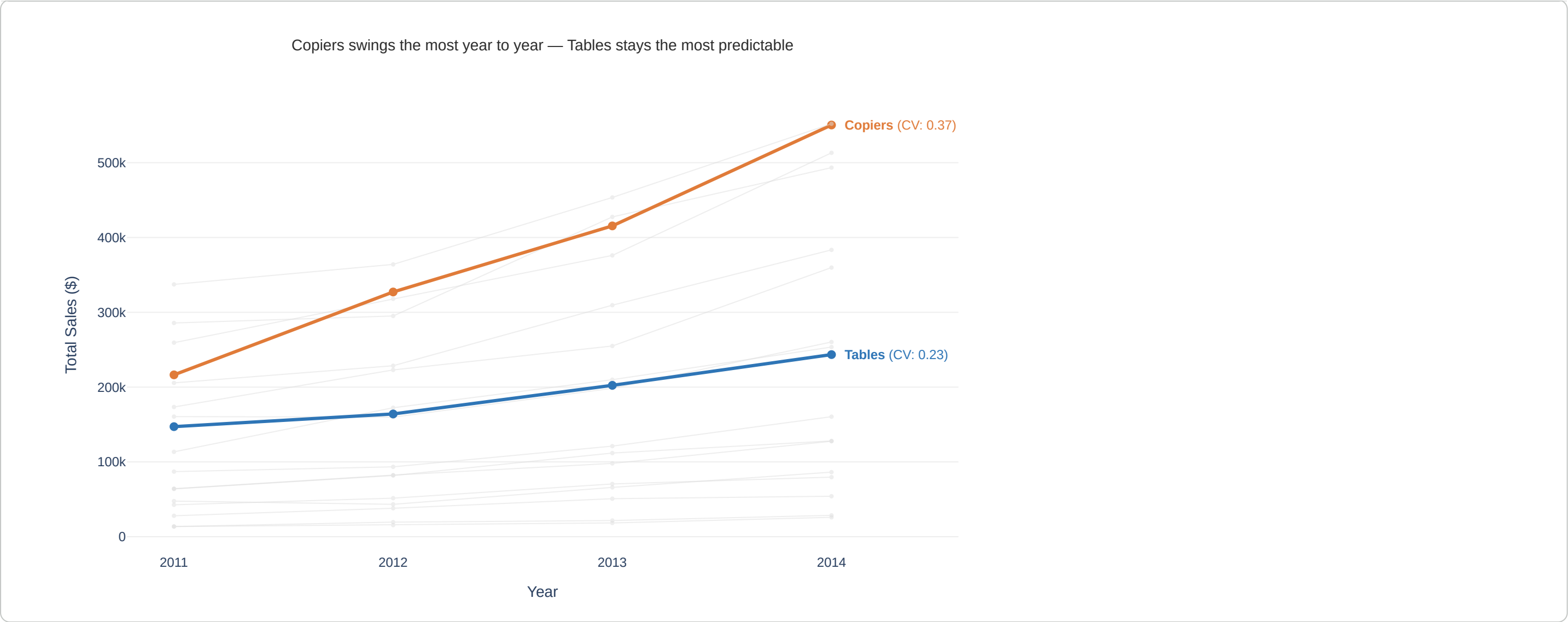
# — Step 1: Plotly Express base chart —————
fig = px.line(
    sub_year, x='Year', y='Sales', color='Sub-Category',
    color_discrete_map=color_map,
    labels={'Sales': 'Total Sales ($)'},
    markers=True
)

# — Step 2: Line weight – highlighted sub-categories thicker —————
fig.update_traces(line=dict(width=1), marker=dict(size=4), opacity=0.5, showlegend=False)
fig.update_traces(line=dict(width=3), marker=dict(size=8), opacity=1,
    selector=dict(name=most_volatile))
fig.update_traces(line=dict(width=3), marker=dict(size=8), opacity=1,
    selector=dict(name=most_stable))

# — Step 3: Direct labels for the two highlighted lines only —————
for sc, color in [(most_volatile, '#E07B39'), (most_stable, '#2E75B6')]:
    pt = sub_year[(sub_year['Sub-Category'] == sc) & (sub_year['Year'] == sub_year['Year'].max())]
    fig.add_annotation(
        x=pt['Year'].values[0], y=pt['Sales'].values[0],
        text=f'<b>{sc}</b> (CV: {cv[sc]:.2f})', showarrow=False,
        xanchor='left', xshift=10,
        font=dict(color=color, size=12, family='Arial')
    )

# — Step 4: Customisation —————
fig.update_layout(
    title=dict(
        text=f'{most_volatile} swings the most year to year – {most_stable} stays the most predictable',
        font=dict(family='Arial', size=14, color='#2C2C2C'),
        x=0.5, xanchor='center', y=0.95
    ),
    font=dict(family='Arial', size=12),
    plot_bgcolor='white', paper_bgcolor='white',
    xaxis=dict(title='Year', showgrid=False, tickmode='array', tickvals=sub_year['Year'].unique()),
    yaxis=dict(gridcolor='EEEEEE', title='Total Sales ($)', zeroline=False),
    margin=dict(l=70, r=70, t=80, b=50),
    height=550, width=900
)

fig.show()
```

Q8 — Sub-Category Sales Volatility (Multi-Line with Highlight)

Question: Which product sub-categories show the most volatile (seasonal) sales patterns across years, and which are stable?

Chart type & why A multi-line chart tracks each sub-category's sales trajectory across years — appropriate since Year is a genuine trend, not a discrete category.

Volatility metric Coefficient of variation (CV = std / mean) across the 4 years per sub-category — this normalises for scale, so a small sub-category and a large one are compared fairly on how *erratic* their growth is, not just their absolute swings.

Highlight technique Grey-and-highlight with 17 total lines: only the most volatile (orange) and most stable (blue) sub-categories are coloured; all 15 others recede to thin grey lines, each still directly labelled at its endpoint for reference without resorting to a legend.

Decluttering No vertical gridlines, light horizontal gridlines only, white background — consistent with the rest of the notebook.

Insight Copiers is the most volatile sub-category (CV: 0.37) — its sales trajectory swings hardest year to year — while Tables is the most stable (CV: 0.23), growing at a steady, predictable pace despite being the sub-category that loses the most money (see Q4).

Q9: Does a faster/costlier ship mode actually correlate with higher order value or profitability?

```
# Cap Sales at 95th percentile so extreme orders don't compress the box plot
p95 = df['Sales'].quantile(0.95)
df_cap = df[df['Sales'] <= p95].copy()

ship_summary = (df.groupby('Ship Mode')
                .agg(Avg_Sales=('Sales', 'mean'), Avg_Shipping_Cost=('Shipping Cost', 'mean'))
                .reset_index())

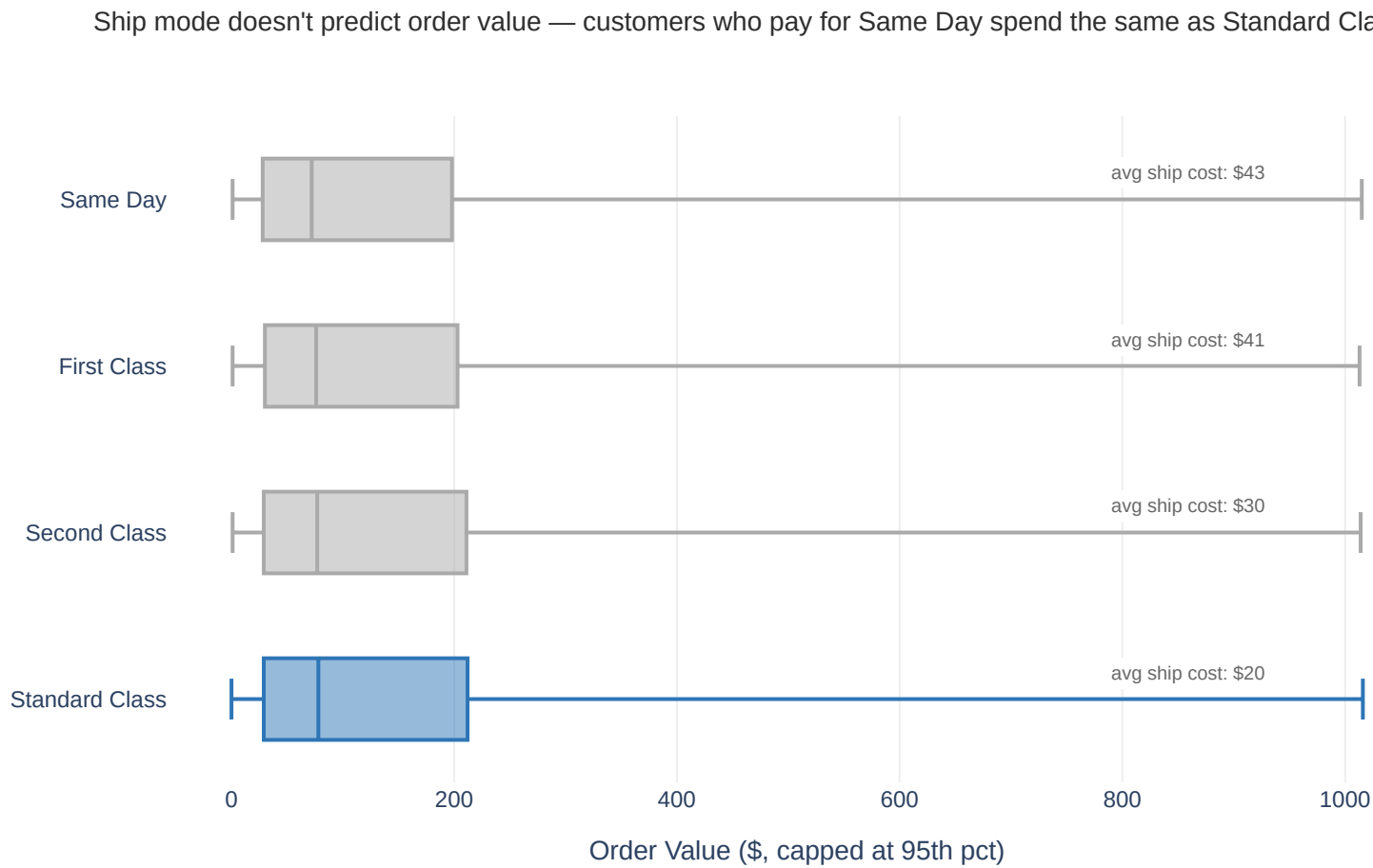
# Colour: highlight Standard Class (cheapest to ship) — everything else grey
color_map = {sm: ('#2E75B6' if sm == 'Standard Class' else '#AAAAAA')
             for sm in df_cap['Ship Mode'].unique()}

# — Step 1: Plotly Express base chart —————
fig = px.box(
    df_cap,
    x='Sales', y='Ship Mode',
    color='Ship Mode',
    color_discrete_map=color_map,
    points=False,
    category_orders={'Ship Mode': ['Same Day', 'First Class', 'Second Class', 'Standard Class']},
    labels={'Sales': 'Order Value ($, capped at 95th pct)', 'Ship Mode': ''}
)

# — Step 2: Annotate the real differentiator — shipping cost, not order value —
for _, row in ship_summary.iterrows():
    fig.add_annotation(
        x=p95 * 0.92, y=row['Ship Mode'],
        text=f"avg ship cost: ${row['Avg_Shipping_Cost']:.0f}",
        showarrow=False, font=dict(size=10, color='#666666', family='Arial'),
        xanchor='right', yshift=14, # added — lifts text above the whisker line
        bgcolor='white', borderpad=2 # added — clean background so line doesn't cross text
    )

# — Step 3: Customisation —————
fig.update_layout(
    title=dict(
        text="Ship mode doesn't predict order value — customers who pay for Same Day spend the same as Standard Class",
        font=dict(family='Arial', size=14, color='#2C2C2C'),
        x=0.5, xanchor='center', y=0.95
    ),
    font=dict(family='Arial', size=12),
    plot_bgcolor='white', paper_bgcolor='white',
    xaxis=dict(gridcolor='#EEEEEE', title='Order Value ($, capped at 95th pct)'),
    yaxis=dict(showgrid=False),
    showlegend=False,
    margin=dict(l=100, r=70, t=80, b=70),
    height=500, width=850
)

fig.show()
```



Q9 — Ship Mode vs Order Value (Box Plot)

Question: Does a faster/costlier ship mode actually correlate with higher customer order value or profitability?

Chart type & why A box plot is used to compare the full distribution (not just averages) of order value across the four ship modes — this matters because it's easy for a faster mode to *look* different on the mean alone while the underlying spread is identical.

Outlier handling Order values capped at the 95th percentile so the bulk of the distribution is visible — disclosed directly in the axis label.

Highlight Standard Class (the cheapest, most common mode) is coloured blue; the three faster/costlier modes recede to grey — since the finding is that speed doesn't buy anything extra, the "default" option is the one worth drawing the eye to.

Annotation Direct-labelled each ship mode's average shipping *cost* next to its box — this is the variable that actually differs, so naming it directly prevents the viewer from assuming order value must differ too just because cost does.

Insight Ship mode has no meaningful relationship with order value — all four modes cluster around the same median (~85). *The only real difference is shipping cost itself: Same Day costs 43 to ship on average vs ~\$20 for Standard Class, yet customers paying for speed spend no more per order.*

Q10: Within the most profitable category, which sub-category's profitability changed most dramatically between the first and last year of data?

```
top_category = df.groupby('Category')['Profit'].sum().idxmax()
sub_cat_change = (df[df['Category'] == top_category]
                  .groupby(['Sub-Category', 'Year'])['Profit']
                  .sum().reset_index())

two_years = sub_cat_change[sub_cat_change['Year'].isin([sub_cat_change['Year'].min(),
                                                       sub_cat_change['Year'].max()])].copy()

two_years['Year'] = two_years['Year'].astype(str)

# Identify the sub-category with the biggest absolute change for the title/annotation
pivot10 = sub_cat_change.pivot(index='Sub-Category', columns='Year', values='Profit')
biggest_mover = (pivot10[pivot10.columns.max()] - pivot10[pivot10.columns.min()]).idxmax()

# Colour: blue = grew, grey = grew less dramatically — all still shown for context
color_map = {sc: ('#2E75B6' if sc == biggest_mover else '#AAAAAA')
             for sc in two_years['Sub-Category'].unique()}

# — Step 1: Plotly Express base chart —————
fig = px.line(
    two_years.sort_values('Year'), x='Year', y='Profit', color='Sub-Category',
    color_discrete_map=color_map, markers=True,
    labels={'Profit': '', 'Year': ''}
)

# — Step 2: Labels at both ends of each line —————
for sc in two_years['Sub-Category'].unique():
    d = two_years[two_years['Sub-Category'] == sc].sort_values('Year')
    is_top = sc == biggest_mover
    fig.update_traces(
        selector=dict(name=sc),
        mode='lines+markers+text',
        text=[f'{d["Profit"].iloc[0]:.0f}', f'{sc}<br>{d["Profit"].iloc[1]:.0f}'],
        textposition=['middle left', 'middle right'],
        textfont=dict(size=11 if is_top else 9,
                      color='#2E75B6' if is_top else '#AAAAAA', family='Arial'),
        # hovertemplate='<b>{fullData.name}</b><br>Year: {x}<br>Profit: ${y:,.2f}<extra></extra>', # added — 2 decimal places
        line=dict(width=3 if is_top else 1),
        showlegend=False
    )

# — Step 3: Customisation —————
years = sorted(two_years['Year'].unique())
fig.update_layout(
    title=dict(
        text=f'{biggest_mover} profit grew the most within {top_category} — nearly quadrupling from {years[0]} to {years[1]}',
        font=dict(family='Arial', size=14, color='#2C2C2C'),
        x=0.5, xanchor='center', y=0.95
    ),
    font=dict(family='Arial', size=12),
    xaxis=dict(tickvals=[0, 1], ticktext=years, showgrid=False, range=[-0.6, 1.8]),
    yaxis=dict(showgrid=False, showticklabels=False, title=''),
    plot_bgcolor='white', paper_bgcolor='white',
    margin=dict(l=40, r=10, t=90, b=40),
    height=550, width=850
)

fig.show()
```

Copiers profit grew the most within Technology — nearly quadrupling from 2011 to 2014

